

QazconHub — Детальное документ-описание проекта

Полное техническое описание цифровой платформы логистического терминала ТОО «QazconHub».

Документ охватывает архитектуру, стек технологий, схему базы данных, все API-эндпоинты, ролевую модель, структуру фронтенда, PWA-возможности, безопасность и развёртывание.

1. Обзор проекта

QazconHub — это современное **PWA-приложение** (Progressive Web App) для комплексного управления логистическим терминалом. Платформа автоматизирует ключевые операционные процессы терминала:

-  **Контейнерные операции** — учёт, размещение и отслеживание контейнеров.
-  **Завоз и вывоз** — документы терминала по бумажным формам со сквозной нумерацией, жизненным циклом статусов, пропуском по документу и вывозом по завозу.
-  **Расходные накладные** — шапка с доверенностью и получателем, строки с контейнерами, грузом, количеством и складом, создание вывоза по строке.
-  **Справочники** — склады, контрагенты и ответственные лица для форм документов.
- **Железнодорожные пути (ППЖТ)** — управление путями и вагонами.
-  **Спецтехника** — учёт кранов, погрузчиков и тягачей.
-  **Транспорт и пропускной режим** — въезд/выезд, OCR-распознавание номеров, шлагбаумы, пропуска.
-  **Электронная очередь** — запись водителей и назначение слотов.
-  **Заявки клиентов** — создание и обработка заявок.
-  **Журнал действий и отчёты** — аудит и PDF-отчёты.

Ключевая особенность — **единая роль-ориентированная панель управления**: один и тот же интерфейс автоматически адаптируется под права конкретного пользователя (13 ролей), показывая только доступные ему разделы.

2. Архитектура

Платформа построена по классической **клиент-серверной** модели с **монолитным** бэкендом и **SPA-подобным** фронтендом.



Поток запроса

1. Пользователь входит через `POST /api/auth` → получает **JWT-токен** (7 дней).
2. Токен сохраняется в `localStorage` (`token`, `user`) и передаётся в заголовке `Authorization: Bearer <token>`.
3. Каждый защищённый эндпоинт проходит через middleware `authRequired` (проверка JWT) и, при необходимости, `roleRequired(...)` (проверка роли).
4. Сервер выполняет операции с SQLite и, для изменяющих операций, рассылает **realtime-события** через Socket.IO всем подключённым клиентам в комнате `terminal`.
5. Фронтенд получает событие и автоматически перерисовывает текущий раздел (если он открыт).

3. Технологический стек

Слой	Технология	Назначение
Бэкенд	Node.js (≥18), Express 4	HTTP-сервер, REST API
База данных	SQLite + better-sqlite3	Локальная файловая БД (WAL, FK)
Авторизация	jsonwebtoken (JWT), bcryptjs	Токены и хэширование паролей
Реальное время	Socket.IO 4	Живые обновления интерфейса
Защита	express-rate-limit	Ограничение частоты запросов
Загрузка файлов	multer 2	Загрузка фото (до 5 МБ, изображения)
PDF-отчёты	PDFKit	Генерация отчётов по контейнерам/вагонам
Файловая система	fs-extra	Создание директории загрузок
Фронтенд	HTML5, CSS3, Vanilla JS	Без фреймворков, лёгкий и быстрый
PWA	Service Worker, Web App Manifest	Офлайн-кэш, установка на устройство

Слой	Технология	Назначение
Деплой	Docker, Render (Blueprint), Fly.io	Контейнеризация и облачный хостинг
Тесты	Node.js smoke + RBAC (112 проверок)	Изолированный прогон на временной БД

4. Структура проекта

delivery-pwa/	# корень проекта (QazconHub Terminal)
├─ server/	
├─ server.js	# API-сервер (Express + Socket.IO), ~1090 строк
├─ db.js	# схема БД + начальные данные (seed)
└─ db/	
└─ terminal.db	# файл базы данных SQLite (+ WAL/SHM)
├─ public/	
├─ index.html	# страница входа
├─ dashboard.html	# роль-ориентированная панель управления
├─ manifest.json	# PWA-манифест
├─ sw.js	# Service Worker (кэш v7)
├─ icon.png	# иконка приложения
└─ css/	
└─ style.css	# стили (адаптивные, 5 брейкпоинтов)
└─ js/	
├─ api.js	# модуль API, авторизация, утилиты, словари
├─ dashboard.js	# ядро панели + модули (часть 1)
├─ dashboard-views.js	# модули (часть 2) + инициализация
└─ dashboard-docs.js	# документооборот (часть 3): завоз/вывоз, накладные, справочники
└─ uploads/	# загруженные фото (создаётся автоматически)
├─ test/	
├─ api-test.js	# smoke + RBAC-тест API (112 проверок)
└─ run-isolated.sh	# изолированный прогон: свой сервер на :8099 + временная БД
├─ tools/	
├─ cleanup-test-data.js	# очистка рабочей БД от артефактов автотестов (dry-run/--apply)
└─ build-project-pdf.js	# сборка PROJECT.pdf из PROJECT.md (npm run docs:pdf)
├─ package.json	# зависимости и скрипты (start, test, test:direct, db:clean-test, docs:pdf)
├─ package-lock.json	
├─ Dockerfile	# контейнеризация
├─ .dockerignore	# исключения сборки образа (локальная БД, node_modules, скриншоты)
├─ render.yaml	# Blueprint для Render (ветка qazconhub, autoDeploy)
├─ fly.toml	# конфигурация Fly.io
├─ .env.example	# шаблон переменных окружения
├─ .gitignore	
├─ DEPLOY.md	# пошаговая инструкция деплоя на Render
├─ README.md	# краткое описание и быстрый старт
├─ PROJECT.md	# настоящий документ
└─ PROJECT.pdf	# PDF-версия описания проекта

5. База данных (SQLite)

Файл БД: `server/db/terminal.db`. Включены режимы **WAL** (`journal_mode = WAL`) и **внешние ключи** (`foreign_keys = ON`).

5.1 Таблицы

`users` — пользователи и роли

Поле	Тип	Описание
<code>id</code>	INTEGER PK	Идентификатор
<code>login</code>	TEXT UNIQUE	Логин
<code>pass_hash</code>	TEXT	Хэш пароля (bcrypt)
<code>name</code>	TEXT	ФИО / название
<code>role</code>	TEXT	Роль (13 значений)
<code>terminal</code>	TEXT	Терминал (по умолч. «Основной»)
<code>active</code>	INTEGER	Активен (1/0)
<code>created_at</code>	TEXT	Дата создания

`containers` — контейнеры

Поле	Тип	Описание
<code>id</code>	INTEGER PK	Идентификатор
<code>number</code>	TEXT UNIQUE	Номер контейнера (напр. TCLU1234567)
<code>size</code>	TEXT	Размер (20/40/45 фт)
<code>type</code>	TEXT	Тип (Гружёный/Порожний/Рефрижератор)
<code>status</code>	TEXT	Статус (см. словарь)
<code>zone / row / stack / tier</code>	TEXT	Размещение: зона/ряд/ярус/уровень
<code>client</code>	TEXT	Клиент
<code>cargo</code>	TEXT	Груз
<code>weight</code>	REAL	Вес (т)
<code>created_at / updated_at</code>	TEXT	Метки времени

`tracks` — железнодорожные пути

Поле	Тип	Описание
<code>id</code>	INTEGER PK	Идентификатор
<code>name</code>	TEXT UNIQUE	Название (Путь №1...)
<code>capacity</code>	INTEGER	Вместимость (вагонов)
<code>zone</code>	TEXT	Зона
<code>status</code>	TEXT	Статус (свободен)

wagons — вагоны

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
number	TEXT UNIQUE	Номер вагона
cargo	TEXT	Груз
owner	TEXT	Владелец
track_id	INTEGER FK→tracks	Путь
status	TEXT	Статус (см. словарь)
direction	TEXT	Направление
dest_station	TEXT	Станция назначения
shipper	TEXT	Отправитель
operation	TEXT	Операция (прибытие/выгрузка/...)
created_at / updated_at	TEXT	Метки времени

equipment — спецтехника

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
name	TEXT	Название (Кран КК-1...)
type	TEXT	Тип (Козловой кран и т.д.)
plate	TEXT	Госномер
status	TEXT	Статус (свободна/работает/на ремонте)
driver	TEXT	Водитель
lat / lon	REAL	Координаты (GPS)
fuel	REAL	Топливо (%)
hours	REAL	Моточасы
created_at	TEXT	Дата создания

vehicles — транспорт (въезд/выезд)

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
plate	TEXT	Госномер
type	TEXT	Тип (Грузовой/Легковой/...)
driver	TEXT	Водитель
purpose	TEXT	Цель визита
direction	TEXT	Направление (въезд)
status	TEXT	Статус (на_территории/выехал)
time_in	TEXT	Время въезда
time_out	TEXT	Время выезда
container_id	INTEGER FK→containers	Связанный контейнер

Поле	Тип	Описание
created_at	TEXT	Дата создания

 requests — заявки клиентов

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
client_id	INTEGER FK→users	Клиент
client_name	TEXT	Имя клиента
type	TEXT	Тип (контейнер/вагон/хранение/другое)
container_number	TEXT	Номер контейнера
cargo	TEXT	Груз
weight	REAL	Вес (т)
status	TEXT	Статус (см. словарь)
date_slot / time_slot	TEXT	Желаемый слот
comment	TEXT	Комментарий
created_at / updated_at	TEXT	Метки времени

 queue — электронная очередь

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
plate	TEXT	Госномер
driver	TEXT	Водитель
purpose	TEXT	Цель
slot	TEXT	Назначенный слот
status	TEXT	Статус (см. словарь)
created_at	TEXT	Дата создания

 passes — пропуска

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
code	TEXT UNIQUE	Сквозной номер пропуска как в журнале терминала (000020129)
vehicle_id	INTEGER FK→vehicles	Транспорт
operation_id	INTEGER FK→operations	Документ завоза/вывоза, по которому выдан пропуск
plate	TEXT	Госномер
type	TEXT	Тип (разовый/постоянный)
status	TEXT	Статус (активен/использован)
doc_date / created_at	TEXT	Дата документа / создания

audit_log — журнал действий

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
user_id	INTEGER	Пользователь
user_name	TEXT	Имя пользователя
action	TEXT	Код действия (container:create и т.д.)
details	TEXT	Детали
created_at	TEXT	Дата создания

warehouses — склады (справочник для документов)

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
name	TEXT UNIQUE	Название склада (КНС, СПРЕЙДЕР, ППЖТ...)
code	TEXT	Короткий код
active	INTEGER	Активен (1/0)
created_at	TEXT	Дата создания

counterparties — контрагенты (справочник для документов)

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
name	TEXT UNIQUE	Название организации/лица
kind	TEXT	Роль в документе: организация / контрагент / собственник / получатель / перевозчик
bin	TEXT	БИН
contact	TEXT	Контакт
active	INTEGER	Активен (1/0)
created_at	TEXT	Дата создания

operations — завоз и вывоз (главный документ терминала)

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
doc_no	TEXT UNIQUE	Номер документа (завоз 06860 , вывоз 000004855)
kind	TEXT	Тип: завоз / вывоз
op_date	TEXT	Дата документа
org	TEXT	Организация (шапка документа)
counterparty_id / owner_id / recipient_id	INTEGER FK→counterparties	Контрагент / собственник / получатель
warehouse_id	INTEGER FK→warehouses	Склад
movement	TEXT	Движение («Завоз порожний», «Вывоз гружёный»...)
transport_mode	TEXT	Вид транспорта (Автотранспортом / Железной дорогой)

Поле	Тип	Описание
status	TEXT	Статус документа (Черновик / Оформлен / Завершён / Отменён)
cargo_status	TEXT	Статус груза по образцу (Порожний / Грузенный / Грузенный (реф))
transferred	INTEGER	Отметка «Передан» (1/0), заполняется у вывоза
container_id	INTEGER FK→containers	Контейнер
container_category	TEXT	Категория контейнера (20 футовый / 40 футовый...)
seal_no	TEXT	Номер пломбы
plate / driver	TEXT	Госномер / водитель
wagon_id / wagon_kind	INTEGER FK→wagons / TEXT	Вагон и его вид
ignore_wagon	INTEGER	Не учитывать вагон (1/0)
basis_id	INTEGER FK→operations	Документ-основание (завоз для вывоза)
pass_id	INTEGER FK→passes	Пропуск, введённый по документу
invoice_id	INTEGER FK→invoices	Расходная накладная, к которой привязан вывоз
responsible_id	INTEGER FK→users	Ответственное лицо
comment	TEXT	Примечание
created_at / updated_at	TEXT	Метки времени

 invoices — расходные накладные (шапка)

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
doc_no	TEXT UNIQUE	Номер накладной (000004855)
doc_date	TEXT	Дата документа
org	TEXT	Организация
recipient_id / owner_id	INTEGER FK→counterparties	Код получателя / собственник
proxy_no / proxy_from / proxy_to / proxy_person	TEXT	Доверенность: номер, срок действия, кому выдана
proxy_individual	INTEGER	Доверенность выдана физлицу (1/0)
basis_doc	TEXT	Основание (договор/заявка)
status	TEXT	Статус (см. словарь)
responsible_id	INTEGER FK→users	Ответственное лицо
comment	TEXT	Примечание
created_at / updated_at	TEXT	Метки времени

 invoice_items — строки накладной

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
invoice_id	INTEGER FK→invoices	Накладная

Поле	Тип	Описание
operation_id	INTEGER FK→operations	Документ завоза (основание строки)
container_id / container_number	INTEGER FK→containers / TEXT	Контейнер
cargo_kind	TEXT	Вид груза
category	TEXT	Категория контейнера
unit / qty	TEXT / REAL	Единица измерения и количество
warehouse_id	INTEGER FK→warehouses	Склад
comment	TEXT	Примечание

1234

doc_seq — счётчики сквозной нумерации документов

Поле	Тип	Описание
scope	TEXT PK	Область нумерации: <code>завоз</code> , <code>вывоз</code> , <code>накладная</code> , <code>пропуск</code>
last_no	INTEGER	Последний использованный номер

5.2 Начальные данные (seed)

При первом запуске `db.js` автоматически создаёт (только если таблицы пусты):

- **13 пользователей** (см. таблицу ролей в разделе 7).
- **4 железнодорожных пути**: Путь №1–2 (Зона А, 25 ваг.), Путь №3–4 (Зона Б, 20 ваг.).
- **4 единицы спецтехники**: Кран КК-1, Кран КК-2, Погрузчик П-1, Тягач Т-1.
- **Склады терминала**: КНС, СПРЕЙДЕР, ППЖТ, Склад №1 (из образцов бумажных форм).
- **Контрагенты**: собственники и получатели контейнеров (ТОО, АО, физлица).
- **Счётчики нумерации** `doc_seq` : завоз продолжается с `06860` , вывоз и накладная — с

`000004855` , пропуск — с `000020129` .

6. Словари статусов

Статусы хранятся в БД в **машинном** виде (латиницей, нижний регистр) и отображаются пользователю в **человекочитаемом** виде через словари в `api.js` .

Статусы контейнеров (CONTAINER_STATUS)

Код	Отображение
на_терминале	На терминале
ожидает_вывоза	Ожидает вывоза
вывезен	Вывезен
в_обработке	В обработке

Статусы вагонов (WAGON_STATUS)

Код	Отображение
на_пути	На пути
выгрузка	Выгрузка

Код	Отображение
погрузка	Погрузка
готов_к_отправке	Готов к отправке
отправлен	Отправлен

 Статусы заявок (REQUEST_STATUS)

Код	Отображение
новая	Новая
подтверждена	Подтверждена
выполняется	Выполняется
завершена	Завершена
отклонена	Отклонена

 Статусы очереди (QUEUE_STATUS)

Код	Отображение
ожидание	Ожидание
вызван	Вызван
на_территории	На территории
завершён	Завершён

 Статусы спецтехники (EQUIPMENT_STATUS)

Код	Отображение
свободна	Свободна
работает	Работает
на_ремонте	На ремонте

 Статусы транспорта (VEHICLE_STATUS)

Код	Отображение
на_территории	На территории
выехал	Выехал

 Статусы документов завоза/вывоза (OPERATION_STATUS)

Код	Отображение
черновик	Черновик
оформлен	Оформлен
завершён	Завершён
отменён	Отменён

Статусы расходных накладных (INVOICE_STATUS)

Код	Отображение
черновик	Черновик
оформлена	Оформлена
закрита	Закрита

Дополнительные справочники документов: OPERATION_KINDS (завоз/вывоз) и PASS_TYPE (разовый/постоянный). Названия справочных записей в таблицах документов подставляются по id через refName() , поэтому в выпадающих списках отображаются имена, а не номера.

Каждому статусу соответствует CSS-класс бейджа (через statusClass()): st-ok , st-done , st-warn , st-info , st-new , st-assigned , st-onway , st-cancelled .

7. Роли и права доступа

7.1 Список ролей (13)

Роль (код)	Отображение	Логин	Пароль
admin	Администратор	admin	admin123
director	Директор	director	director123
dispatcher	Диспетчер	dispatcher	disp123
ppjt	Диспетчер ППЖТ	ppjt	ppjt123
receiver	Приёмосдатчик	receiver	recv123
crane	Крановщик	crane	crane123
store	Кладовщик	store	store123
guard	Охрана	guard	guard123
customs	Таможенный отдел	customs	customs123
finance	Расчётный отдел	finance	fin123
shift	Начальник смены	shift	shift123
client	Клиент	client	client123
driver	Водитель	driver	driver123

7.2 Доступ к разделам панели (матрица NAV)

Матрица задана в `dashboard.js` (объект NAV). Каждая роль видит только свои разделы.

Раздел	admin	director	dispatcher	ppjt	receiver	crane	store	guard	customs	finance	shift	client	driver
 Обзор	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	—	—
 Контейнеры	✓	—	✓	—	✓	✓	✓	—	✓	—	✓	—	—
 Пути и вагоны	✓	—	—	✓	✓	—	—	—	—	—	✓	—	—
 Спецтехника	✓	—	✓	—	—	✓	—	—	—	—	✓	—	—

Раздел	admin	director	dispatcher	ppjt	receiver	crane	store	guard	customs	finance	shift	client	driver
 Транспорт	✓	—	✓	—	—	—	—	✓	—	—	✓	—	—
 Заявки	✓	—	✓	—	—	—	—	—	✓	✓	✓	✓	—
 Очередь	✓	—	✓	—	—	—	—	✓	—	—	✓	✓	✓
 Завоз	✓	—	✓	✓	✓	—	✓	✓	—	—	✓	—	—
 Вывоз	✓	—	✓	✓	✓	—	✓	✓	—	—	✓	—	—
 Расходные накладные	✓	—	✓	—	—	—	✓	—	—	✓	✓	—	—
 Справочники	✓	—	✓	—	—	—	—	—	—	—	✓	—	—
 Пропускной режим	✓	—	✓	—	—	—	—	✓	—	—	✓	—	—
 Журнал действий	✓	✓	—	—	—	—	—	—	—	—	✓	—	—
 Отчёты	✓	✓	—	—	—	—	—	—	—	✓	✓	—	—

7.3 Права на операции (RBAC на сервере)

Права на создание/редактирование проверяются на сервере через `roleRequired(...)` :

Ресурс	Создание	Редактирование
Контейнеры	admin, dispatcher, receiver, guard, shift	admin, dispatcher, receiver, crane, shift
Пути	admin, ppjt	—
Вагоны	admin, ppjt, receiver	admin, ppjt, receiver, shift
Спецтехника	admin, dispatcher, shift	admin, dispatcher, crane, shift
Транспорт (въезд)	admin, guard, dispatcher, shift	admin, guard, dispatcher, shift (выезд)
Заявки	client, admin, dispatcher	admin, dispatcher, shift, client (только свои)
Очередь	driver, client, admin, dispatcher	admin, dispatcher, guard, shift
Пропуска	admin, guard, dispatcher, shift	—
Завоз / вывоз	admin, dispatcher, receiver, guard, shift, ppjt	admin, dispatcher, receiver, guard, shift, ppjt
Пропуск по документу	admin, guard, dispatcher, shift	—
Расходные накладные	admin, dispatcher, finance, shift	admin, dispatcher, finance, shift
Строки накладной и вывоз по строке	admin, dispatcher, shift	admin, dispatcher, shift
Справочник складов	admin, dispatcher	—
Справочники (чтение)	все авторизованные	—
Справочник контрагентов	admin, dispatcher, shift	—
OCR / шлагбаум	admin, guard, dispatcher, shift	—
Журнал (просмотр)	admin, director, shift	—
Отчёты (PDF)	admin, director, finance, shift	—

8. API-эндпоинты (REST)

Базовый URL: `/api`. Все эндпоинты, кроме `POST /api/auth`, требуют заголовок `Authorization: Bearer <JWT>`.

8.1 Аутентификация и справочники

Метод	Путь	Роли	Описание
POST	<code>/api/auth</code>	— (публичный, rate-limit 30/15мин)	Вход: <code>{login, pass}</code> → <code>{token, user}</code>
GET	<code>/api/roles</code>	все	Справочник ролей (код → название)
GET	<code>/api/stats</code>	все	Сводная статистика терминала
GET	<code>/api/dictionaries</code>	все	Справочники для форм документов: склады, контрагенты, ответственные, значения полей
GET	<code>/api/warehouses</code>	все	Склады
POST	<code>/api/warehouses</code>	admin, dispatcher	Добавить склад (дубликат названия → 409)
GET	<code>/api/counterparties</code>	все	Контрагенты
POST	<code>/api/counterparties</code>	admin, dispatcher, shift	Добавить контрагента (неизвестная роль → «контрагент»)

`GET /api/stats` возвращает: `containers`, `containersFull`, `wagons`, `wagonsOnTrack`, `equipment`, `eqFree`, `vehicles`, `queue`, `requestsNew`, а также метрики документооборота: `operationsOpen`, `operationsIn`, `operationsOut`, `invoicesToday`, `passesToday`.

8.2 Контейнеры

Метод	Путь	Роли	Описание
GET	<code>/api/containers</code>	все	Список контейнеров
POST	<code>/api/containers</code>	admin, dispatcher, receiver, guard, shift	Создать контейнер
PATCH	<code>/api/containers/:id</code>	admin, dispatcher, receiver, crane, shift	Обновить (статус, размещение, клиент, груз, вес)

8.3 Пути (ППЖТ)

Метод	Путь	Роли	Описание
GET	<code>/api/tracks</code>	все	Пути + вагоны на них + заполненность
POST	<code>/api/tracks</code>	admin, ppjt	Создать путь

8.4 Вагоны

Метод	Путь	Роли	Описание
GET	<code>/api/wagons</code>	все	Список вагонов (с названием пути)
POST	<code>/api/wagons</code>	admin, ppjt, receiver	Добавить вагон (прибытие)
PATCH	<code>/api/wagons/:id</code>	admin, ppjt, receiver, shift	Обновить (путь, статус, направление, станция)

8.5 Спецтехника

Метод	Путь	Роли	Описание
GET	<code>/api/equipment</code>	все	Список техники

Метод	Путь	Роли	Описание
POST	/api/equipment	admin, dispatcher, shift	Добавить технику
PATCH	/api/equipment/:id	admin, dispatcher, crane, shift	Обновить (статус, водитель, топливо, моточасы, GPS)

8.6 Транспорт

Метод	Путь	Роли	Описание
GET	/api/vehicles	все	Список транспорта
POST	/api/vehicles	admin, guard, dispatcher, shift	Регистрация въезда
PATCH	/api/vehicles/:id/exit	admin, guard, dispatcher, shift	Оформление выезда

8.7 Заявки клиентов

Метод	Путь	Роли	Описание
GET	/api/requests	все	Список (клиент видит только свои)
POST	/api/requests	client, admin, dispatcher	Создать заявку
PATCH	/api/requests/:id	admin, dispatcher, shift, client	Обновить (клиент — только свою)

8.8 Электронная очередь

Метод	Путь	Роли	Описание
GET	/api/queue	все	Список очереди
POST	/api/queue	driver, client, admin, dispatcher	Встать в очередь
PATCH	/api/queue/:id	admin, dispatcher, guard, shift	Обновить (статус, слот)

8.9 Пропуска, OCR, шлагбаумы

Метод	Путь	Роли	Описание
GET	/api/passes	admin, guard, dispatcher, shift	Список пропусков
POST	/api/passes	admin, guard, dispatcher, shift	Создать пропуск (сквозной номер 000020129); можно связать с документом через operation_id
POST	/api/ocr/recognize	admin, guard, dispatcher, shift	Распознать номер → доступ/запрет
POST	/api/barrier/:action	admin, guard, dispatcher, shift	Открыть/закрыть шлагбаум (open / close)

8.10 Журнал и отчёты

Метод	Путь	Роли	Описание
GET	/api/audit	admin, director, shift	Журнал действий (последние 200)
GET	/api/report/containers	admin, director, finance, shift	PDF-отчёт по контейнерам
GET	/api/report/wagons	admin, director, finance, shift	PDF-отчёт по вагонам

8.11 Загрузка файлов

Метод	Путь	Роли	Описание
POST	/api/upload-photo	все (авторизованные)	Загрузка фото (multipart, поле photo) → {url}

8.12 Завоз и вывоз (документы терминала)

Метод	Путь	Роли	Описание
GET	/api/operations	все	Журнал документов. Параметры: kind (завоз/вывоз), status, q (номер, контейнер, госномер, водитель)
GET	/api/operations/:id	все	Карточка документа + связанные pass, invoice и строки накладных
POST	/api/operations	admin, dispatcher, receiver, guard, shift, ppjt	Создать документ (kind обязателен, номер — из doc_seq)
PATCH	/api/operations/:id	admin, dispatcher, receiver, guard, shift, ppjt	Изменить документ: шапка целиком (org, контрагенты, склад, контейнер и категория, cargo_status, способ тр-ки, movement, plate, driver, seal_no, вагон, ответственный, comment, status, ignore_wagon, transferred)
POST	/api/operations/:id/pass	admin, guard, dispatcher, shift	«Ввести пропуск» по документу: пропуск создаётся и связывается с завозом/вывозом

Особенности:

- Номер документа выдаётся из doc_seq: завоз — 5 цифр (06860), вывоз — 9 (000004855).
- basis_id принимается только на **завоз** (вывоз оформляется по завозу, иначе 400).
- Неизвестный статус игнорируется — остаётся текущее значение.
- PATCH применяет только переданные поля: cargo_status и transferred пишутся как есть, transferred актуален для вывоза (в форме отображается чекбоксом «Передан»).
- op_date возвращается и сохраняется со временем (например 17.09.2026 12:21:45).
- Повторный POST /api/operations/:id/pass не создаёт второй пропуск, а возвращает уже выданный (200).
- Документ без госномера пропуск не получает (400).

8.13 Расходные накладные

Метод	Путь	Роли	Описание
GET	/api/invoices	все	Список накладных (со счётчиком и суммой строк); ?full=1 — со строками
GET	/api/invoices/:id	все	Накладная с шапкой и строками
POST	/api/invoices	admin, dispatcher, finance, shift	Создать накладную; строки можно передать сразу массивом items
PATCH	/api/invoices/:id	admin, dispatcher, finance, shift	Изменить шапку (получатель, доверенность, статус)
POST	/api/invoices/:id/items	admin, dispatcher, shift	Добавить строку (контейнер, категория, склад подтягиваются из документа завоза)

Метод	Путь	Роли	Описание
POST	/api/invoices/:id/items/bulk	admin, dispatcher, shift	«Добавить из отпуска»: пакетно создать строки по отмеченным завозам ({operation_ids: []} → {added}, повторные завозы пропускаются)
DELETE	/api/invoices/:id/items/:itemId	admin, dispatcher, shift	Удалить строку
POST	/api/invoices/:id/items/:itemId/outbound	admin, dispatcher, shift	«Создать вывоз по текущей строке»: вывоз наследует контейнер, категорию, склад, контрагентов и завоз-основание

Особенности:

- Шапка правится через PATCH: получатель и собственник, «Через кого» (proxy_no, proxy_from, proxy_to, proxy_person), чекбокс «Физ лицо» (proxy_individual), basis_doc, статус, ответственный, комментарий.
- Все операции со строками и выпуск вывоза из строки доступны ролям admin, dispatcher и shift.

9. Реальное время (Socket.IO)

Сервер инициализирует Socket.IO поверх HTTP-сервера с CORS origin: '*'.

События клиент → сервер

- terminal:join — клиент входит в общую комнату terminal и, при передаче роли, в комнату role:<роль>.

События сервер → клиент (emitUpdate)

Сервер рассылает события всем в комнате terminal. Фронтенд перерисовывает раздел, только если он сейчас открыт:

Событие	Триггер	Раздел
container:update	создание/изменение контейнера	Контейнеры
wagon:update	добавление/изменение вагона	Пути и вагоны
equipment:update	изменение техники	Спецтехника
vehicle:update	въезд/выезд транспорта	Транспорт
request:update	создание/изменение заявки	Заявки
queue:update	изменение очереди	Очередь
pass:update	оформление пропуска, в том числе по документу завоза/вывоза	Пропускной режим
barrier:update	открытие/закрытие шлагбаума	Пропускной режим (индикатор шлагбаума)
operation:update	создание/изменение завоза, вывоза, пропуск по документу, вывоз из строки накладной	Завоз / Вывоз
invoice:update	создание накладной, добавление/удаление строки	Расходные накладные
dictionary:update	изменение складов и контрагентов	Справочники (сброс кэша справочников)

Модуль `dashboard-docs.js` переиспользует соединение, созданное в `dashboard-views.js` (`window.__dashboardSocket`), чтобы не открывать второе Socket.IO-подключение на страницу.

10. Фронтенд

10.1 Страница входа — `index.html`

- Форма логина/пароля, отправка на `POST /api/auth`.
- При успехе токен и пользователь сохраняются в `localStorage`, редирект на `/dashboard.html`.
- Если уже авторизован — сразу переход в панель.
- Сетка демо-доступов для быстрого входа.

10.2 Панель управления — `dashboard.html`

Структура:

- **Сайдбар** (`.sidebar`): бренд, навигация (`#sideNav`), блок пользователя с кнопкой выхода.
- **Основная область** (`.main`): топбар (кнопка меню `≡`, заголовок, индикатор «Онлайн», часы) и контент (`#content`).
- **Оверлей** (`.sidebar-overlay`) — для мобильного меню.
- **Модальное окно** (`.modal-overlay` / `.modal`).

10.3 Модуль API — `api.js`

- `API.getToken/setToken/getUser/setUser/logout` — работа с `localStorage`.
- `API.request/get/post/patch` — обёртки над `fetch` с авторизацией и обработкой 401.
- `API.uploadPhoto` — загрузка файла через `FormData`.
- `connectSocket()` — подключение Socket.IO с токеном.
- Словари статусов, `ROLE_LABELS`, `statusLabel`, `statusClass`, `esc` (XSS-защита), `fmtDate`, `toast`.

10.4 Ядро панели — `dashboard.js` (часть 1)

- Проверка авторизации (редирект на `/` при отсутствии).
- Объект `NAV` (14 разделов с ролями) и вычисление `allowedNav` по роли.
- Часы в реальном времени.
- `buildNav()`, `switchView()`, `openModal()`, `closeModal()`, `render()`.
- Модули: **Обзор** (включая карточки документооборота), **Контейнеры**, **Пути и вагоны**,

Спецтехника, Транспорт.

- Экспорт ядра в `window.__dashboardCore` (включая `VIEWS`).

10.5 Дополнительные модули — `dashboard-views.js` (часть 2)

- Модули: **Заявки**, **Очередь**, **Пропускной режим** (OCR + шлагбаум), **Журнал действий**, **Отчёты**.
- Инициализация: `buildNav()`, первичный `render()`.
- Подключение Socket.IO и обработка realtime-событий.
- Кнопка выхода.
- **Мобильное меню**: открытие/закрытие с оверлеем, Escape, свайпы, блокировка прокрутки, сброс при `resize`.

10.6 Документооборот — `dashboard-docs.js` (часть 3)

- Модули: **Завоз, Вывоз, Расходные накладные, Справочники.**
- Кэш справочников на клиенте, сбрасывается по событию `dictionary:update`.
- Реестры документов: колонки как в образцах терминала (Способ тр-ки, Категория, Статус

груза, Номер машины, Вагон, Пропуск, Расходная, Статус документа), фильтр по статусу, поиск (q) и кнопка «Статус» для быстрой смены состояния документа.

- Карточка документа — редактируемая форма в порядке полей образца: заголовок

«Заявка на завоз: Завоз 06861 от ...», поля «Номер машины», «Способ тр-ки», «Статус» (груз), «Ответственный», чекбоксы «Не учитывать вагон» / «Передан», кнопки «ОК» / «Записать» / «Заккрыть» и выпуск связанных документов («Ввести пропуск», «Ввести расходную накладную», «Создать вывоз по завозу»). Роли без права правки получают карточку только для чтения.

- Карточка расширена до 720px (на мобильных — вся ширина), заголовок и блок кнопок

залипают (`position: sticky`) при прокрутке, поэтому «ОК / Записать / Заккрыть» всегда доступны; при ширине ≤ 600 px кнопки выстраиваются в один столбик.

- Карточка накладной использует ту же оболочку: широкая таблица строк прокручивается

внутри модального окна, не разрывая шапку и кнопки.

- Накладные: редактируемая шапка (включая «Через кого» и чекбокс «Физ лицо»),

строки с кнопками «Создать вывоз по текущей строке» и «Удалить», подбор строк («+ Строка», «Добавить из завоза», «Добавить из отпуска»).

- Справочники: добавление складов и контрагентов без перезагрузки страницы.
- Realtime-подписки на `operation:update`, `invoice:update`, `pass:update`, `dictionary:update`

через уже открытое соединение (`window.__dashboardSocket`).

10.7 Стили — `style.css`

- Тёмно-синяя тема (`#0f2a4a` → `#0b2138`), акцент `#ffb300`.
- Сайдбар 250px, сетка статистики `auto-fill minmax(200px, 1fr)`, таблицы с горизонтальным скроллом (`.table-wrap`).
- Модальные карточки: заголовок (`.modal > h3`) и футер (`.modal-actions`) — `sticky`

с фоном и рамкой, тело карточки — единственная прокручиваемая область (`.modal { overflow: hidden }, .modal-body { overflow: auto }`).

- **Адаптивность — 5 брейкпоинтов:**

- ≤ 1100 px — планшет (компактная сетка).
 - ≤ 900 px — мобильное меню (off-canvas сайдбар 270px, кнопка ≡).
 - ≤ 600 px — мобильный (одна колонка).
 - ≤ 480 px — вход (адаптация).
 - ≤ 380 px — очень маленькие экраны (сайдбар на всю ширину).
 - `.grid-2 > * { min-width: 0 }` — защита от горизонтального переполнения таблиц.
-

11. PWA (Progressive Web App)

11.1 Манифест — `manifest.json`

- `name`: «QazconHub — Логистический терминал», `short_name`: «QazconHub».
- `display`: `standalone`, `theme_color`: `#0f2a4a`, `background_color`: `#f0f4f8`.
- Иконки 192×192 и 512×512 (`icon.png`, `purpose`: `any maskable`).
- `start_url`: `/`, `lang`: `ru`.

11.2 Service Worker — `sw.js`

- Кэш: `qazconhub-cache-v7` (версия обновляется при изменениях статики).
- **Статические ресурсы** кэшируются при установке: `/`, `index.html`, `dashboard.html`, `style.css`, `api.js`, `dashboard.js`, `dashboard-views.js`, `dashboard-docs.js`, `manifest.json`.
- **Активация**: удаляются все старые кэши (кроме текущего), `clients.claim()`.
- **Стратегия кэширования**:
- **API и Socket.IO** (`/api`, `/socket.io`) — **не кэшируются** (всегда сеть).
- **Навигация** — `network-first` с фолбэком на кэш (офлайн-доступ к панели).
- **Статика** — `cache-first` с фоновым обновлением.
- Версия кэша инкрементируется при каждом изменении статики (сейчас `v7`), что форсирует

обновление у клиентов — без этого вернувшиеся пользователи получают старый интерфейс из `cache-first` кэша.

12. Безопасность

Платформа реализует многоуровневую защиту:

Мера	Реализация
Хэширование паролей	<code>bcrypt (cost 10)</code> — пароли не хранятся в открытом виде
JWT-авторизация	Токены со сроком 7 дней, подпись секретным ключом
Ролевой доступ (RBAC)	<code>roleRequired(...)</code> на каждом изменяющем эндпоинте
Rate limiting	API: 500 запросов/15 мин; вход: 30 попыток/15 мин
XSS-защита	Экранирование вывода через <code>esc()</code> на фронтенде
Аудит действий	Каждое значимое действие пишется в <code>audit_log</code>
Ограничение загрузок	Только изображения, максимум 5 МБ (<code>multer</code>)
Обработка ошибок	Централизованный <code>middleware</code> , без утечки деталей
Секреты в окружении	<code>JWT_SECRET</code> через <code>.env</code> (не в коде)

Ключевые `middleware`-защиты

- `authRequired` — проверяет заголовок `Authorization: Bearer <token>`, расшифровывает JWT и кладёт `req.user`. При недействительном токене — `401`.
- `roleRequired(...roles)` — проверяет, что роль `req.user.role` входит в разрешённый список. Иначе — `403`.
- `apiLimiter` — глобальный лимит на все `/api`-запросы.

- `authLimiter` — усиленный лимит на эндпоинт входа (защита от brute-force).

13. Развёртывание

13.1 Локальный запуск

```
# Установка зависимостей
npm install

# Настройка окружения (опционально)
cp .env.example .env

# Запуск сервера (порт 8080)
npm start
```

Сервер доступен на `http://localhost:8080`. База данных `server/db/terminal.db` создаётся автоматически при первом запуске вместе с демо-данными.

13.2 Переменные окружения (`.env`)

Переменная	По умолчанию	Описание
<code>JWT_SECRET</code>	<code>qazconhub-terminal-secret-change-in-prod</code>	Секрет подписи JWT. Обязателен при <code>NODE_ENV=production</code> — без него сервер завершается с <code>[FATAL]</code>
<code>PORT</code>	<code>8080</code>	Порт HTTP-сервера
<code>NODE_ENV</code>	—	<code>production</code> включает строгую проверку <code>JWT_SECRET</code>
<code>DB_PATH</code>	<code>server/db/terminal.db</code>	Путь к файлу SQLite (изолированные прогоны тестов, внешний диск)

13.3 Docker

`Dockerfile` собирает образ на базе `node:20-alpine`:

```
docker build -t damulogistics .
docker run -p 8080:8080 damulogistics
```

Особенности:

- Устанавливаются инструменты сборки (`python3` `make` `g++`) для нативного модуля `better-sqlite3`.
- `npm ci --omit=dev` — только продакшен-зависимости (с использованием кэша слоёв Docker).
- Создаётся директория `public/uploads`.
- `NODE_ENV=production`, порт `8080`.

13.4 Fly.io

`fly.toml` — конфигурация облачного деплоя:

```
fly launch
fly deploy
```

Конфигурация: приложение `qazconhub-terminal`, регион `ams`, HTTP-сервис на порту `8080` с `force_https`, авто-стоп/старт машин, 1 виртуальная машина (512 МБ RAM, 1 CPU).

13.5 Render (Blueprint)

`render.yaml` описывает сервис как код: тип `web`, runtime `docker`, план `free`, регион `frankfurt`, health-check /, `autoDeploy: true`, ветка деплоя — `branch: qazconhub`.

- `JWT_SECRET` генерируется Render автоматически (`generateValue: true`), `NODE_ENV=production` задаётся в манифесте.
- В репозитории несколько проектов: разворачивать нужно ветку `qazconhub`, а не `master`, иначе Render не найдёт Blueprint.
- При пуше в ветку `qazconhub` сервис пересобирается автоматически; интерфейс обновляется у клиентов после поднятия версии кэша в `public/sw.js`.
- Явное `branch: qazconhub` фиксирует ветку деплоя, поэтому результат не зависит от ветки по умолчанию в репозитории; при создании Blueprint ветку всё равно нужно выбрать вручную — Render читает манифест из выбранной ветки.
- Сервис, созданный вне Blueprint, Render не «подхватывает» автоматически: чтобы получить управляемый Blueprint с нужным именем, создайте новый Blueprint-проект, а старый сервис удалите.
- Бесплатный тариф: сервис засыпает после 15 минут простоя (холодный старт 30–60 с), файловая система эфемерная — БД пересоздаётся вместе с seed-данными при каждом перезапуске, для постоянного хранения нужен платный Disk.

Подробная пошаговая инструкция — в [DEPLOY.md](#).

13.6 Описание проекта (PROJECT.pdf)

Единый источник описания — [PROJECT.md](#). PDF-версия для отправки заказчику собирается из него:

```
npm run docs:pdf # нужен Chrome/Chromium
CHROME_PATH=/путь/к/chrome npm run docs:pdf # если браузер не найден автоматически
```

[tools/build-project-pdf.js](#) конвертирует Markdown в HTML (заголовки, 45 таблиц, код-блоки, списки, ссылки) и печатает его в A4-вёрстку через headless-браузер. Скрипт не тянет внешних зависимостей, поэтому обновление описания сводится к правке `PROJECT.md` и одному запуску.

14. Тестирование и проверка

14.1 API-тесты — `api-test.js`

```
npm test # изолированно: run-isolated.sh поднимает сервер на :8099 с временной БД в /tmp
```

Тест создаёт записи и не удаляет их за собой, поэтому прямой запуск против рабочей БД (`server/db/terminal.db`, порт `:8080`) блокируется fail-fast проверкой в начале `api-test.js`. Другие режимы:

```
BASE_URL=https://<хост> npm run test:direct # против внешнего стенда
ALLOW_DIRTY_DB=1 npm run test:direct # осознанно в рабочую БД
npm run db:clean-test # dry-run: показать артефакты тестов
npm run db:clean-test -- --apply # удалить их (с бэкапом terminal.db)
```

112 проверок, все зелёные. Что покрывается:

- Авторизация: вход под всеми ролями, отказ при неверном пароле и без токена.
- Все модули: контейнеры, пути, вагоны, техника, транспорт, заявки, очередь, пропуска,

OCR/шлагбаум, отчёты (PDF), загрузка фото.

- **Документооборот:** нумерация (06860 , 000004855 , 000020129), карточка завоза с

фильтрами и поиском, поля cargo_status и transferred (включая отметку «Передан» через PATCH), пропуск по документу (повторный вызов идемпотентен), накладная со строками, подтягивание контейнера/категории/склада из завоза, пакетный подбор строк /items/bulk (повторные завозы пропускаются), «создать вывоз по строке».

- **RBAC:** 20+ негативных проверок (client, driver, guard, finance не могут того, что

не входит в их права) и позитивные проверки доступов.

- Валидации: неизвестный kind или неизвестная роль контрагента не ломают документ

(подставляются безопасные значения), дубликат склада → 409.

14.2 Браузерная проверка (Playwright)

Платформа тестировалась через Playwright (браузерная автоматизация) на трёх размерах экрана:

Размер	Проверка
375px (мобильный)	Сайдбар off-canvas, кнопка ≡, статистика в 1 колонку, таблицы со скроллом, нет горизонтального переполнения
768px (планшет)	Сайдбар off-canvas (270px), кнопка ≡, статистика в 4 колонки
1280px (десктоп)	Сайдбар статичный, кнопка меню скрыта, сетки в несколько колонок

Проверены сценарии:

- Вход под разными ролями (admin — 14 разделов, guard — 4 раздела).
- Открытие/закрытие мобильного меню (кнопка, пункт меню, Escape, оверлей, свайпы).
- Блокировка/разблокировка прокрутки тела при открытом меню.
- Разделы **Завоз**, **Вывоз**, **Расходные накладные**, **Справочники**: реестры, фильтры,

формы создания документа и накладной со строками, карточка документа с кнопкой «Ввести пропуск».

- Редактируемая карточка документа (поля «Номер машины / Способ тр-ки / Статус груза /

Ответственный», чекбоксы «Не учитывать вагон» и «Передан», кнопки «ОК / Записать / Закрыть») и накладной («Через кого», чекбокс «Физ лицо», «Добавить из отпуска», «Создать вывоз по текущей строке»); сохранение через «Записать» проверено и на API, и в браузере.

- Широкая карточка (720px) с залипающими заголовком и футером: кнопки «ОК / Записать /

Закрыть» остаются на экране при прокрутке длинной формы (в т.ч. на 375px — столбиком).

- Пустые реестры после очистки БД: «Завоз на терминал (0)» и «Расходные накладные (0)»

отрисовывают заглушку без ошибок в консоли; создание нового завоза продолжило нумерацию с 06860 — значит счётчики doc_seq откатаны без пропусков в номерах.

- Отсутствие ошибок в консоли браузера.

14.3 Гигиена данных — `cleanup-test-data.js`

Автотесты пишут в БД, поэтому появился инструмент точечной очистки рабочих данных:

```
npm run db:clean-test          # dry-run: печатает, что будет удалено
npm run db:clean-test -- --apply # удалить (сначала делает бэкап terminal.db)
```

Правила поиска: контейнеры `TEST*`, вагоны `WAG*`, техника/склады/пути/контрагенты с префиксами «ТЕСТ-», пропуска `P*` и очередь `Q*/D*/T*`, заявки `REQ*`, документы, созданные после 15.09.2026 (дата появления первого прогона), и записи журнала `audit_log`, относящиеся к удалённым сущностям. После удаления счётчики `doc_seq` откатываются к последнему «настоящему» номеру, поэтому следующие документы получают номера без дыр. Бэкап (`server/db/terminal.db.bak-<время>`) позволяет откатиться, если что-то было удалено лишнее.

15. Возможности для развития

- Подключение **реального OCR** (вместо симуляции распознавания номера).
- Интеграция с **GPS-трекингом** спецтехники (поля `lat / lon` уже есть в схеме).
- **Уведомления** (push) для клиентов и водителей.
- **Мультитерминальная** поддержка (поле `terminal` у пользователей).
- Управление пользователями (CRUD) через админ-панель.
- Расширение отчётности (экспорт в Excel, графики).
- **Печатные формы** завоза, вывоза и расходной накладной в PDF по образцам бумажных бланков.
- Обмен документами с учётной системой (**1C / ERP**) по API.
- Резервное копирование базы данных.

16. Заключение

QazconHub — полнофункциональная, готовая к эксплуатации PWA-платформа управления логистическим терминалом. Она объединяет в едином адаптивном интерфейсе все ключевые операционные процессы: от учёта контейнеров и вагонов до пропускного режима с OCR и электронной очереди, а также полный документооборот терминала — завоз и вывоз по бумажным формам, расходные накладные со строками и справочники складов и контрагентов. Благодаря ролевой модели (13 ролей), realtime-синхронизации через Socket.IO, PWA-возможностям (офлайн, установка) и адаптивному дизайну (мобильные/планшетные/десктопные экраны) платформа пригодна для использования как на стационарных рабочих местах терминала, так и в полевых условиях сотрудниками, клиентами и водителями.

Качество подтверждается автотестами: 112 проверок API (авторизация, RBAC по 13 ролям, все модули, документооборот и справочники) выполняются в изолированном окружении — свой сервер на порту :8099 и временная БД в `/tmp`, поэтому рабочая база не загрязняется. Дополнительно есть инструмент очистки рабочей БД от артефактов прогонов (`tools/cleanup-test-data.js`) с бэкапом и откатом счётчиков нумерации `doc_seq`.